

SmartCourse

AI-Powered Course Recommendation System

FYP Viva Preparation Guide

Viva Schedule

Microsoft Teams viva: Thursday, April 30, 2026, 11:00 AM - 11:10 AM. Keep answers short, structured, and connected to the running project.

Project Metadata

- Group ID: F25PROJECTEC4D8
- Roll No: BC240419615
- Supervisor: Muhammad Bilal
- Project root: /mnt/f/my-projects/smartcourse-ai-app
- Generated: 2026-04-29 00:34

1. Quick Viva Opening

My FYP is SmartCourse, an AI-powered course recommendation system. It helps learners find relevant courses based on their learning goals. The system compares two recommendation approaches: TF-IDF, which focuses on keyword similarity, and Neural sentence embeddings, which understand semantic meaning. It is built with Flask, uses a Bootstrap frontend, stores search history and saved recommendations in SQLite, and uses trained model artifacts for fast recommendations.

How to speak in the viva

Start with the problem, then explain your solution, then the architecture, then the algorithm. If a question is difficult, say 'In the current prototype...' and explain the current scope plus a future improvement.

2. Problem and Solution

Students often struggle to choose suitable courses from a large catalog. Manual searching is slow, and basic keyword search may miss useful courses when the wording is different. SmartCourse lets users type natural language goals such as 'I want to learn Python for data science' and returns ranked course suggestions.

Main Features

- Multi-page web UI: Home, Recommendations, Dashboard, and About.
- Natural language input for learning goals.

- Model selection: TF-IDF, Neural, or Hybrid comparison.
- Score bars and course cards for recommended courses.
- Search history and saved recommendations stored in SQLite.
- Side-by-side model comparison dashboard.
- Operational logging in instance/smartcourse.log.

3. Technology Stack

- Backend: Python, Flask, Flask-SQLAlchemy, Flask-Cors.
- Frontend: Bootstrap 5, HTML templates, vanilla JavaScript, Fetch API.
- AI/NLP: scikit-learn, spaCy, sentence-transformers, joblib.
- Database: SQLite.
- Dataset: raw CSV has 8,092 rows; cleaned CSV currently has 4,939 usable course records.
- Model artifacts: models/tfidf_recommender.joblib and models/neural_recommender.joblib.

4. Directory Structure

Think of this project like a web app where Flask replaces Express/Laravel, templates replace frontend views, and the AI model files act like prebuilt search indexes.

```
smartcourse-ai-app/  
|-- app.py  
|-- requirements.txt  
|-- README.md  
|-- smartcourse/  
|   |-- __init__.py  
|   |-- api.py  
|   |-- views.py  
|   |-- config.py  
|   |-- extensions.py  
|   |-- models/  
|   |-- services/  
|   |-- data/  
|   '-- utils/  
|-- templates/  
|-- static/  
|-- data/  
|-- models/  
|-- instance/  
|-- scripts/  
|-- tests/  
 '-- project-documents/
```

What Each Folder Means

- app.py: the main entry point, similar to server.js or index.js. It starts the Flask server.
- requirements.txt: Python dependency list, similar to package.json dependencies.
- smartcourse/: the main backend package. It contains routes, services, configuration, database models, and AI model code.

- templates/: server-rendered HTML pages such as base.html, home.html, recommend.html, dashboard.html, and about.html.
- static/: frontend assets. main.js handles fetch calls and UI rendering; main.css contains styling.
- data/: raw, cleaned, sample, and evaluation datasets.
- models/: trained AI model artifacts loaded by the app at runtime.
- instance/: runtime files such as SQLite database and logs.
- scripts/: maintenance scripts for preparing data, building models, evaluating models, exporting SQL, and packaging.
- project-documents/: SRS, design, submission, and viva documents.

5. Request Flow

When a learner searches for a course, the app moves through this flow:

```
Browser form
-> static/js/main.js sends fetch('/api/recommend')
-> smartcourse/api.py receives the POST request
-> RecommendationService validates and runs selected model
-> TFIDFRecommender or NeuralRecommender ranks courses
-> HistoryService saves the search session in SQLite
-> JSON response returns to the browser
-> JavaScript renders recommendation cards
```

6. Coding Part Explained

app.py

This is the startup file. It imports create_app() from smartcourse, reads command-line host/port/debug options, and starts the Flask development server.

```
from smartcourse import create_app
app = create_app()

if __name__ == "__main__":
    app.run(host=cli_args.host, port=cli_args.port,
            debug=resolve_debug(cli_args))
```

smartcourse/__init__.py

This is the application factory. It creates the Flask app, loads config, initializes database/CORS, registers UI and API blueprints, configures logging, and creates database tables.

- register_extensions(app): connects SQLAlchemy and CORS.
- register_blueprints(app): attaches UI routes and API routes.
- configure_logging(app): writes logs to instance/smartcourse.log.
- db.create_all(): creates SQLite tables if they do not exist.

smartcourse/views.py

This file contains page routes. These routes do not return JSON; they render HTML templates.

```
/ -> templates/home.html
/recommend -> templates/recommend.html
/dashboard -> templates/dashboard.html
/about -> templates/about.html
```

smartcourse/api.py

This is the REST API controller layer. It receives browser requests, validates input, calls services, and returns JSON.

- POST /api/recommend: gets preference, model, top_n; returns ranked courses and creates a search session.
- GET /api/history: returns recent search sessions and saved recommendations.
- POST /api/save: saves a selected course against a search session.
- DELETE /api/history and DELETE /api/saved: clear stored records.

smartcourse/services/recommendation.py

This is the business logic for recommendations. It lazy-loads model files only when needed, runs the selected model, normalizes scores, and cleans values before sending them to the frontend.

- tfidf_model property loads models/tfidf_recommender.joblib.
- neural_model property loads models/neural_recommender.joblib.
- recommend(preference, model, top_n) runs tfidf, neural, or both.
- _normalize_scores() converts scores into a frontend-friendly 0 to 1 range.

smartcourse/services/history.py

This service handles persistence. It creates search sessions, saves selected recommendations, reads history, and clears records.

smartcourse/models/db_models.py

This file defines the database tables using SQLAlchemy.

- SearchSession stores query text, TF-IDF results, Neural results, timestamp, and related saved recommendations.
- SavedRecommendation stores selected course details, model type, score, and save time.

static/js/main.js

This is the browser-side logic. It listens for form submission, sends API requests with fetch(), renders recommendation cards, saves courses, and loads dashboard data.

7. AI Part Explained Simply

TF-IDF Recommender

TF-IDF is a keyword-based technique. It converts course descriptions into vectors where important words receive higher weights. The user query is preprocessed with spaCy, transformed into the same vector space, and compared against all courses using cosine similarity.

- Good for exact terms such as Python, SQL, React, cloud, or cybersecurity.
- Fast and explainable.
- May miss results when meaning is similar but words are different.

Neural Recommender

The neural model uses sentence-transformers/all-MiniLM-L6-v2. It converts full course descriptions into embeddings that capture meaning. The query is also converted into an embedding, then cosine similarity ranks the closest courses.

- Good for semantic meaning and natural language queries.
- Can match related ideas even when exact words differ.
- More computationally expensive than TF-IDF, so embeddings are precomputed and saved.

Hybrid Mode

Hybrid mode runs both TF-IDF and Neural models and displays them side by side. It is designed for comparison rather than merging both scores into one final ranking.

8. Data Pipeline

The data pipeline prepares messy CSV course data before model training.

- Loads raw course dataset from data/courses_dataset.csv.
- Renames different column names into standard names like course_title, university, difficulty, rating, course_description, and department.
- Removes duplicates and invalid rows.
- Drops missing or very short course descriptions.
- Fills missing department, difficulty, university, and rating values.
- Normalizes text using lowercase conversion, HTML removal, alphabetic cleanup, whitespace cleanup, stopword removal, and lemmatization.
- Saves cleaned output to data/courses_clean.csv.

9. Evaluation

The project includes scripts/evaluate_models.py. It loads evaluation_queries.json and calculates Precision@K, Recall@K, and Hit Rate.

- Precision@K: out of the top K recommended courses, how many are relevant?
- Recall@K: out of all known relevant courses, how many did the model find in top K?
- Hit Rate: did the top K results contain at least one relevant course?

Local environment note

During inspection, evaluation failed because the saved joblib artifacts were built with a different scikit-learn/pandas version than the current virtual environment. The fix is to rebuild models in the current environment using scripts/prepare_data.py and scripts/build_models.py.

10. Likely Instructor Questions

- Why did you use TF-IDF? It is fast, simple, explainable, and strong for keyword-based retrieval.
- Why use sentence transformers? They capture semantic meaning, so related phrases can match even without exact words.
- What is cosine similarity? It measures how close two vectors are by comparing their direction.

- Why Flask? It is lightweight, easy to structure, and suitable for REST APIs plus rendered web pages.
- Why SQLite? It is simple, file-based, and enough for a prototype that stores history and saved recommendations.
- What are limitations? It depends on description quality, has no login/personalization yet, and evaluation data is small.
- How can it improve? Add user accounts, feedback-based ranking, filters, better evaluation data, and cloud deployment.
- What happens if model files are missing? The API returns a service error and asks to rebuild models.

11. Demo Script

- Open the app and show the Home page.
- Go to Recommendations.
- Enter a query such as: I want to learn Python for data science.
- Select Hybrid to show both TF-IDF and Neural results.
- Point out scores, course titles, departments, and descriptions.
- Save one recommendation.
- Open Dashboard and show search history, saved recommendations, and side-by-side model comparison.
- Mention that every search session is stored in SQLite and logged for traceability.

12. Final Memory Card

One-sentence summary

SmartCourse is a Flask-based AI course recommendation system that compares TF-IDF keyword search with neural semantic search over a cleaned course catalog, while storing user searches and saved recommendations in SQLite.

If you forget details during the viva, return to this structure: problem, solution, architecture, algorithm, database, limitations, future work.